Approach Document — Conversational SHL Assessment Recommender

Architecture Overview

This service is a lightweight, stateless FastAPI application that exposes `GET /health` and `POST /chat`. Retrieval is provided by FAISS (semantic search) over a scraped `data/catalog.json` catalog using SentenceTransformers embeddings. A narrow LLM layer (Gemini or OpenAI-compatible) is used only after retrieval to generate concise conversational replies; recommendations themselves are canonical entries drawn from the catalog. This stack was chosen to balance cost, latency, and repeatability: FAISS gives strong recall at low run-time cost while an LLM supplies conversational fluency and decision logic.

Context Engineering & Retrieval

• Catalog ingestion: the catalog is stored as `data/catalog.json` and indexed into `data/catalog.index` (FAISS). Each catalog item stores `name`, `url`, and `test_type` fields — the only fields used for recommendations.

• Bridging the vocabulary gap: the retriever implements abbreviation mappings and targeted heuristic expansions (e.g., `OPQ` → `Occupational Personality Questionnaire`, `G+` → `SHL Verify Interactive G+`). Multi-query composition (`last`, `last_three`, keywords) is used to extract signal from very short user turns.

• Hybrid scoring: semantic candidates come from FAISS; an IDF-weighted lexical scorer and phrase bonuses re-rank candidates. A lightweight coverage injection step promotes canonical slugs for domain tokens. The agent also supports a `DISABLE_RETRIEVER_HEURISTICS` flag for ablation testing.

Agent Design

• Prompting: the system prompt constrains the LLM with strict behavioral rules (Clarify, Recommend, Refine, Compare, Refuse) and instructs the model to return a JSON-compatible response. The prompt includes ranked catalog context to ground any generated rationale and explicitly forbids invented URLs or products.

• Clarify / Recommend / Refine / Compare / Refuse:

• Clarify: the agent asks exactly one targeted question when the user intent is too vague; recommendations remain `[]`.

• Recommend: when the policy indicates readiness, the server attaches a catalog-derived shortlist (1–10 items) and the reply focuses on rationale only.

• Refine: the agent acknowledges constraint changes and the server re-ranks/replaces the shortlist accordingly.

• Compare: the agent uses only catalog fields to compare named assessments; recommendations remain `[]`.

• Refuse: legal, regulatory, or off-topic queries are refused politely with `recommendations=[]`.

• Turn cap & statelessness: the API is stateless; the server enforces a strict 8-turn cap per request (counting both user and assistant messages). The agent also signals `turns_left` in the prompt to avoid unnecessary follow-ups.

Schema & Safety Enforcement (Audit summary)

To ensure the submission passes the automated hard-evals, I audited the code and implemented additional safeguards:

1. Request/response schemas are strict Pydantic models (`app/models.py`) with `extra = "forbid"` to prevent unexpected keys.

2. The `POST /chat` endpoint enforces the 8-turn cap at the API level (counts total messages) and returns a safe clarifying message when exceeded.

3. A response sanitizer (`app/sanitizer.py`) canonicalizes and validates all outgoing responses:

• Converts any dict-like LLM output into a `ChatResponse` instance or returns a safe clarifying fallback.

• Replaces candidate recommendations with canonical catalog entries when their `url` matches `data/catalog.json`.

• Drops any recommendations that do not correspond to the scraped catalog (prevents URL hallucination).

• Ensures `recommendations` is always an array (possibly empty) with at most 10 items, and sets `end_of_conversation` = true iff recommendations are present.

4. The `Agent.respond()` logic was left intentionally conservative: when the internal policy says `recommend_now`, the server overrides any model-specified recommendations with catalog-derived recommendations — the sanitizer further enforces canonicalization.

These changes close likely failure modes where a generative model could return malformed JSON, hallucinated URLs, extra fields, or recommendations outside the catalog.

Evaluation Rigor

• Repro harness: `eval/evaluate_retriever.py` (retriever-only) and `eval/evaluate.py` (end-to-end against `/chat`) replay 10 Markdown conversation traces and compute Mean Recall@10.

• Diagnostics: `scripts/debug_retrieval.py` inspects per-trace ranks and identifies whether expected items are surfaced by retrieval heuristics. `scripts/perturb_eval.py` runs paraphrase/typo perturbations to test robustness.

• Observations: local runs produced Retriever and End-to-end Mean Recall@10 = 1.00 for the provided traces. Perturbation testing revealed sensitivity to very short/generic last-turn queries and punctuation/typos for some traces. These are documented in `debug_perturb_out.txt`.

AI Tool Usage

AI-assisted coding was used for boilerplate generation, iterative debugging, test harness creation, and for writing documentation. All generated code and outputs were reviewed and adapted to meet strict schema and safety requirements; critical logic and design decisions were authored and

validated by the candidate.

Future Improvements

• Cross-encoder reranker: add a supervised cross-encoder to re-score the top-50 semantic candidates for better precision under paraphrase.

• Robust normalization: stronger Unicode and punctuation normalization, and small edit-distance tolerant lexical matching to reduce fragility to typos.

• Production deployment: response caching (Redis), per-request rate-limits, and metrics (per-trace recall, latency, errors) with automated alerting.

This document is intentionally concise; if you would like, I can convert it into a 1–2 page PDF for submission or expand any section with diagrams or example prompts.